

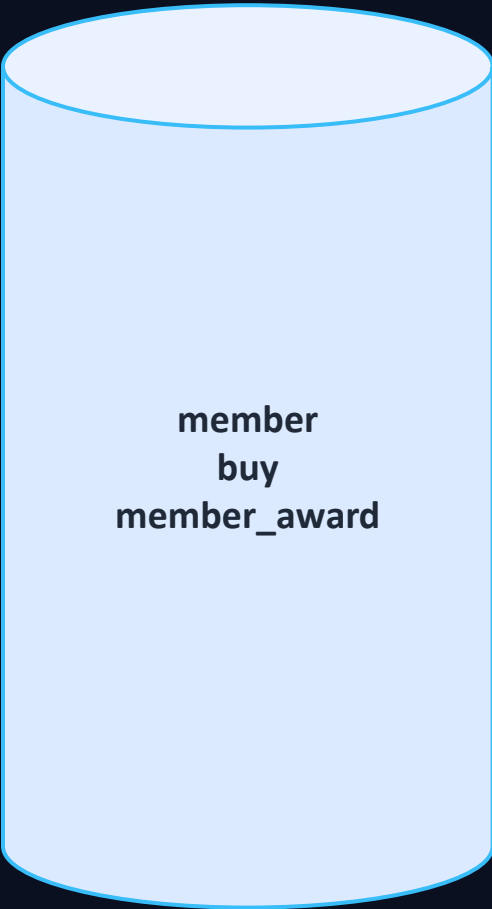
인덱스

검색을 빠르게 만드는 구조와
그 대가를 함께 이해하기

clustered index

secondary index

EXPLAIN 실습



member
buy
member_award

학습 흐름

개념 → 구조 → 장단점 → MySQL 실습

1 인덱스란?

전체를 다 뒤지지 않도록 돕는 검색 구조

2 두 종류

클러스터형 인덱스와 보조 인덱스

3 좋은 인덱스

조회 속도와 쓰기 비용의 균형

4 실습

SHOW INDEX, EXPLAIN, CREATE/DROP INDEX

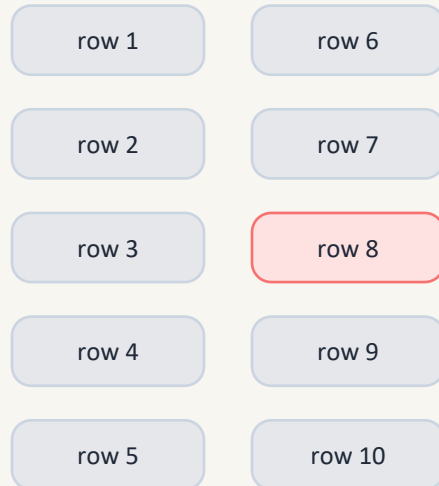
수업 목표

- 기본키와 클러스터형 인덱스의 관계를 설명할 수 있다.
- 고유키와 보조 인덱스의 역할을 구분할 수 있다.
- 불필요한 인덱스가 INSERT/UPDATE/DELETE에 주는 부담을 이해한다.
- EXPLAIN 결과에서 key, rows, type을 보고 인덱스 사용 여부를 확인한다.

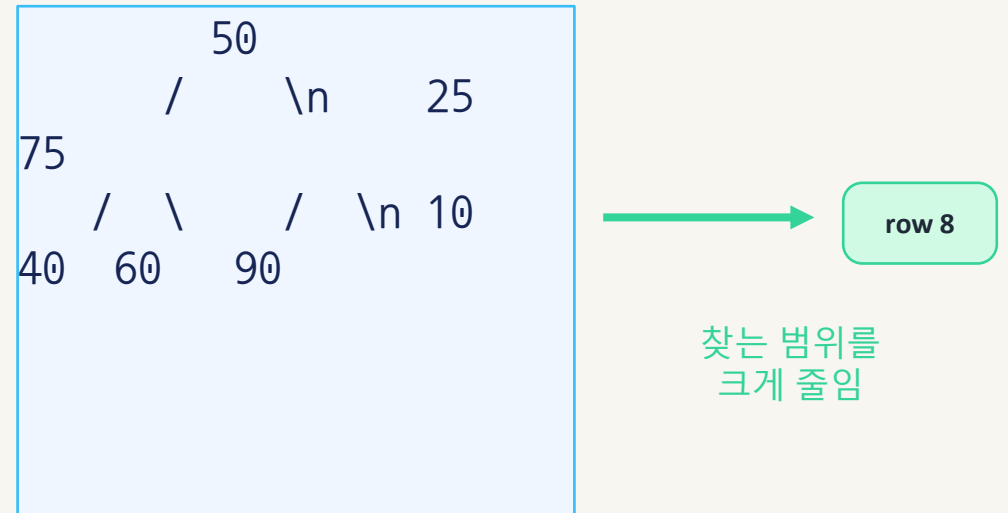
인덱스란 무엇인가?

테이블에서 원하는 행을 빨리 찾기 위한 정렬된 길잡이

인덱스가 없을 때



인덱스가 있을 때



핵심 문장: 인덱스는 데이터를 “저장하는 또 하나의 구조”입니다. 그래서 조회는 빨라질 수 있지만, 쓰기 작업에서는 관리 비용이 발생합니다.

MySQL에서 인덱스는 어떻게 쓰이나?

EXPLAIN으로 “정말 인덱스를 탔는지” 확인한다

```
USE market_db;

EXPLAIN
SELECT mem_id, mem_name, addr
FROM member
WHERE addr = '서울';
```

항목	무엇을 보면 되나?
type	ALL이면 전체 스캔 가능성이 큼 / ref, range면 인덱스 접근 가능성
possible_keys	옵티마이저가 후보로 본 인덱스
key	실제로 선택된 인덱스
rows	확인할 것으로 예상되는 행 수

문법보다 중요한 질문

이 쿼리는 많은 행을 훑는가, 아니면 인덱스로 후보를 좁히는가?

MySQL 공식 문서는 EXPLAIN을 통해 어디에 인덱스를 추가하면 쿼리가 더 빨라질 수 있는지 확인할 수 있다고 설명합니다.

클러스터형 인덱스와 보조 인덱스

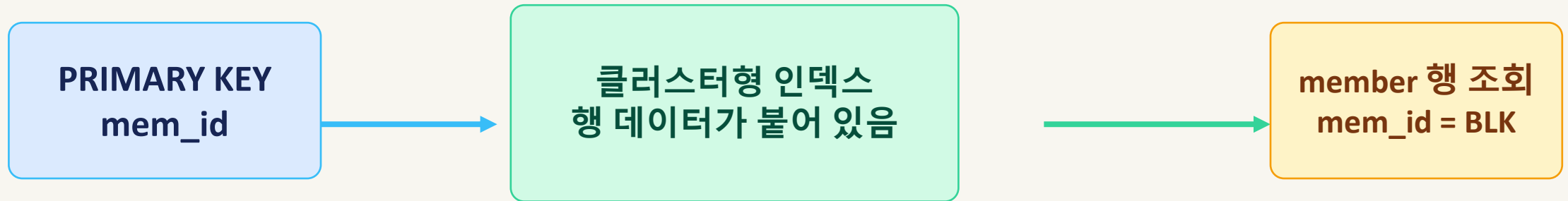
InnoDB 인덱스 이해의 핵심 구분

구분	클러스터형 인덱스	보조 인덱스
대표 예	PRIMARY KEY	INDEX, UNIQUE INDEX
저장 위치	리프 노드에 실제 행 데이터 저장	인덱스 키 + 기본키 값 저장
테이블당 개수	1개	여러 개 가능
찾는 과정	인덱스 검색 = 행 데이터 도착	보조 인덱스 → 기본키 → 클러스터형 인덱스
주요 목적	행의 물리적 저장 기준	특정 조건 검색 보조

MySQL InnoDB에서는 각 테이블에 행 데이터를 저장하는 특별한 인덱스가 있으며, 보통 PRIMARY KEY가 그 역할을 합니다.

기본키와 클러스터형 인덱스의 관계

PRIMARY KEY는 InnoDB에서 테이블의 기준 축이 된다



InnoDB 규칙

- PRIMARY KEY가 있으면 그 키가 클러스터형 인덱스가 된다.
- PRIMARY KEY가 없으면 NOT NULL인 첫 UNIQUE 인덱스를 사용할 수 있다.
- 둘 다 없으면 InnoDB가 숨은 행 ID 기반 클러스터형 인덱스를 만든다.

고유키 UNIQUE란?

중복을 허용하지 않는 보조 식별자

PRIMARY KEY

- 테이블의 대표 식별자
- NULL 불가
- InnoDB에서 클러스터형 인덱스

UNIQUE KEY

- 중복 값 방지
- 여러 개 만들 수 있음
- 대개 보조 인덱스 역할

일반 INDEX

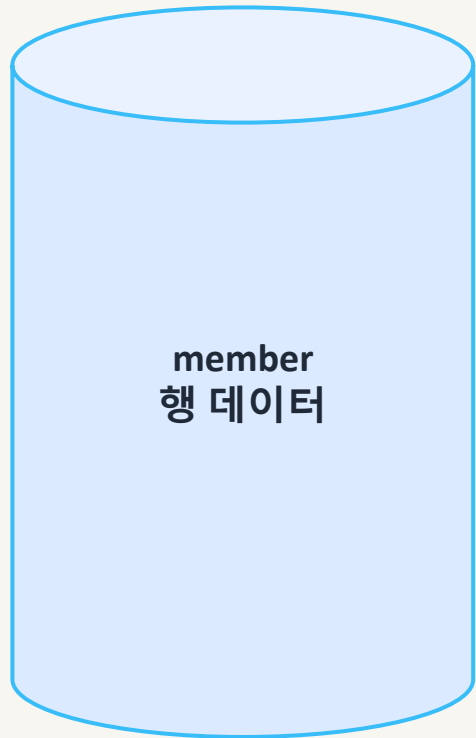
- 중복 허용
- 검색 속도 보조
- 무결성 제약은 아님

```
-- 회원 수상 기록에서 같은 회원이 같은 해 같은 상을  
-- 중복 등록하지 못하게 하고 싶을 때  
CREATE UNIQUE INDEX uq_award_once  
ON member_award(mem_id, award_year, award_name);
```

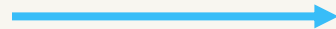
주의: MySQL의 UNIQUE 인덱스는 중복을 막는 동시에 검색용 인덱스로도 사용될 수 있다.

클러스터형 인덱스는 자동으로 정렬된다?

정확히는 “행 데이터가 클러스터형 키 기준으로 조직된다”



PRIMARY KEY(mem_id)



mem_id 기준으로
저장 구조가 조직됨

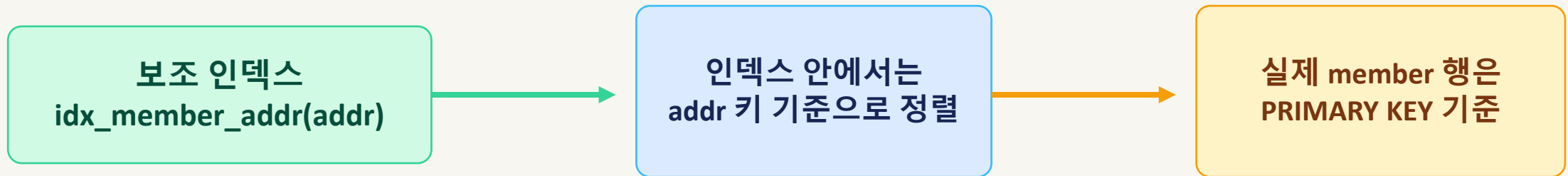


SELECT 결과 순서는
보장하지 않음

```
-- 정렬된 결과가 필요하면 반드시 ORDER BY 사용
SELECT mem_id, mem_name
FROM member
ORDER BY mem_id;
```


보조 인덱스는 정렬되지 않는다?

정확히는 “테이블의 실제 행을 그 기준으로 정렬하지 않는다”



```
CREATE INDEX idx_member_addr ON member(addr);
```

```
EXPLAIN
```

```
SELECT mem_id, mem_name, addr
```

```
FROM member
```

```
WHERE addr = '서울';
```

```
-- 출력 순서가 필요하면
```

```
SELECT mem_id, mem_name, addr
```

```
FROM member
```

```
WHERE addr = '서울';
```

```
ORDER BY addr, mem_id;
```

정리: 보조 인덱스 자체는 B-tree로 정렬되어 있지만, 테이블 행의 물리적 기준을 바꾸지는 않는다.

인덱스의 장점

읽기 성능을 높이고, 불필요한 탐색을 줄인다

빠른 검색

WHERE 조건에 맞는 후보 행을 빠르게 좁힌다

정렬·그룹화 보조

ORDER BY, GROUP BY에서 정렬 비용을 줄일 수 있다

조인 성능 개선

JOIN 조건 컬럼에 인덱스가 있으면 매칭 비용이 줄어든다

무결성 보장

UNIQUE 인덱스는 중복 데이터를 막는다

```
-- 검색, 조인, 정렬이 반복되는 열에 인덱스 후보가 생긴다  
CREATE INDEX idx_buy_mem_prod ON buy(mem_id, prod_name);
```

인덱스의 단점과 불필요한 인덱스의 비용

조회만 빠르게 하는 대신 쓰기과 저장 공간을 사용한다

저장 공간 증가

인덱스도 별도의 자료구조이므로 디스크와 메모리를 사용

쓰기 성능 저하

INSERT/UPDATE/DELETE 때 테이블뿐 아니라 인덱스도 갱신

옵티마이저 혼란

비슷한 인덱스가 많으면 선택지가 늘고 관리가 어려움

중복 관리 비용

거의 같은 복합 인덱스를 여러 개 두면 효과보다 비용이 커짐

좋은 질문

“이 인덱스가 실제 쿼리에서 자주 쓰이는가?”
“이 인덱스가 줄이는 읽기 비용이 늘어나는 쓰기 비용보다 큰가?”

어떤 열에 인덱스를 만들까?

쿼리 조건과 데이터 분포를 함께 본다

좋은 후보

- WHERE에 자주 등장하는 열
- JOIN에 사용되는 외래키 열
- ORDER BY / GROUP BY에 자주 쓰는 열
- 값의 종류가 많아 후보를 크게 줄이는 열

주의 후보

- 거의 검색하지 않는 열
- 값 종류가 매우 적어 잘 걸러지지 않는 열
- 자주 수정되는 열
- 이미 비슷한 복합 인덱스가 있는 열

```
-- 후보 예시
CREATE INDEX idx_member_addr    ON member(addr);
CREATE INDEX idx_member_height ON member(height);
CREATE INDEX idx_buy_mem_id     ON buy(mem_id);
CREATE INDEX idx_award_mem_yr   ON member_award(mem_id, award_year);
```

결론: 인덱스는 “열”이 아니라 “자주 실행되는 쿼리”를 보고 설계한다.

실습 1. 현재 인덱스 확인

SHOW INDEX와 INFORMATION_SCHEMA로 인덱스 목록 보기

```
USE market_db;

SHOW INDEX FROM member;
SHOW INDEX FROM buy;
SHOW INDEX FROM member_award;

SELECT table_name, index_name, non_unique, column_name, seq_in_index
FROM information_schema.statistics
WHERE table_schema = 'market_db'
  AND table_name IN ('member', 'buy', 'member_award')
ORDER BY table_name, index_name, seq_in_index;
```

관찰 포인트

- Key_name이 PRIMARY인지 확인
- Non_unique = 0이면 중복 불가
- Column_name과 Seq_in_index로 복합 인덱스 순서 확인
- Index_type이 보통 BTREE인지 확인

먼저 “이미 있는 인덱스”를 확인해야 중복 인덱스를 만들지 않는다.

실습 2. 인덱스 생성 전후 비교

member.addr 조건 검색을 예로 확인

```
-- 1) 인덱스 생성 전 실행 계획 확인
EXPLAIN
SELECT mem_id, mem_name, addr
FROM member
WHERE addr = '서울';

-- 2) 보조 인덱스 생성
CREATE INDEX idx_member_addr ON member(addr);

-- 3) 다시 실행 계획 확인
EXPLAIN
SELECT mem_id, mem_name, addr
FROM member
WHERE addr = '서울';
```

비교할 값

type

ALL → ref/range?

key

NULL → idx_member_addr?

rows

예상 행 수가 줄었는가?

Extra

Using where 등 참고

데이터 양이 적으면 체감 속도보다 실행 계획 변화에 집중한다.

실습 3. 클러스터형 인덱스 사용

기본키 조건 조회는 클러스터형 인덱스를 직접 탄다

```
-- member 테이블의 mem_id가 PRIMARY KEY라고 가정
SHOW INDEX FROM member;

EXPLAIN
SELECT mem_id, mem_name, addr, height
FROM member
WHERE mem_id = 'BLK';

SELECT mem_id, mem_name, addr, height
FROM member
WHERE mem_id = 'BLK';
```

PRIMARY KEY 검색

행 데이터에
바로 도착

EXPLAIN에서 key = PRIMARY, type = const가 나오면 하나의 기본키 값으로 한 행을 찾는 매우 효율적인 접근입니다.

실습 4. 보조 인덱스 사용

보조 인덱스는 조건 후보를 찾고, 기본키로 행을 가져온다

```
CREATE INDEX idx_member_height ON member(height);

EXPLAIN
SELECT mem_id, mem_name, height
FROM member
WHERE height >= 165;

-- 범위 조건에서는 type이 range로 나타날 수 있다.
```

height 보조 인덱스

mem_id 찾기

member 행 읽기

범위 검색은 인덱스가 특히 유용할 수 있지만, 조건에 맞는 행이 너무 많으면 전체 스캔이 더 낫다고 판단될 수도 있습니다.

실습 5. 조인과 복합 인덱스

buy.mem_id처럼 연결에 자주 쓰는 열을 살펴본다

```
-- 회원별 구매 내역 조회
EXPLAIN
SELECT m.mem_id, m.mem_name, b.prod_name, b.price, b.amount
FROM member AS m
JOIN buy AS b
  ON m.mem_id = b.mem_id
WHERE m.mem_id = 'BLK';

-- 구매 테이블 쪽 검색/조인 보조
CREATE INDEX idx_buy_mem_prod ON buy(mem_id, prod_name);
```



m.mem_id = b.mem_id

복합 인덱스 순서

(mem_id, prod_name)은 mem_id 조건에 먼저 유리합니다.
반대로 prod_name만 조건에 쓰면 효과가 제한될 수 있습니다.

실습 6. UNIQUE 인덱스와 중복 방지

검색 성능 + 데이터 무결성을 함께 확인

```
-- member_award 구조에 award_year, award_name이 있다고 가정
CREATE UNIQUE INDEX uq_award_once
ON member_award(mem_id, award_year, award_name);

-- 같은 회원, 같은 연도, 같은 상 이름은 중복 입력 불가
-- 실제 데이터에 이미 중복이 있으면 인덱스 생성부터 실패한다.

SHOW INDEX FROM member_award;
```

UNIQUE 인덱스 생성 성공

→ 앞으로 같은 조합의 중복 입력 차단

생성 실패

→ 이미 중복 데이터가 존재할 가능성
→ 먼저 중복을 조회·정리해야 함

```
SELECT mem_id, award_year, award_name, COUNT(*)
FROM member_award
GROUP BY mem_id, award_year, award_name
HAVING COUNT(*) > 1;
```

실습 7. 인덱스 삭제와 정리

사용하지 않는 인덱스는 비용만 남길 수 있다

```
-- 인덱스 이름 확인
SHOW INDEX FROM member;
SHOW INDEX FROM buy;

-- 생성한 보조 인덱스 삭제
DROP INDEX idx_member_addr ON member;
DROP INDEX idx_member_height ON member;
DROP INDEX idx_buy_mem_prod ON buy;

-- PRIMARY KEY 삭제는 일반 인덱스 삭제와 다르게 신중해야 함
-- ALTER TABLE member DROP PRIMARY KEY; -- 실습에서는 실행하지 않기
```

삭제 전 확인

- 정말 사용하지 않는가?
- 비슷한 인덱스가 이미 있는가?
- UNIQUE 제약이 필요한 인덱스는 아닌가?
- 외래키 제약과 관련된 인덱스는 아닌가?

실무에서는 삭제 전 운영 쿼리와 제약 조건을 반드시 확인한다.

인덱스 설계 판단 체크리스트

생성하기 전에 5가지를 묻는다

- 1 이 쿼리는 자주 실행되는가?
- 2 WHERE, JOIN, ORDER BY, GROUP BY 중 어디에 쓰이는가?
- 3 인덱스로 후보 행이 충분히 줄어드는가?
- 4 INSERT/UPDATE/DELETE 비용 증가를 감당할 수 있는가?
- 5 기존 인덱스와 중복되지 않는가?

수업 마무리

인덱스는 “빠르게 찾는 길”이지만, 길을 만들고 유지하는 비용까지 함께 설계해야 한다.

수업 진행용 미니 활동

10분 실습 + 5분 토의

활동 A

인덱스 생성 전후 EXPLAIN 비교

```
EXPLAIN SELECT mem_id, mem_name, addr FROM member WHERE addr = '서울';  
CREATE INDEX idx_member_addr ON member(addr);  
EXPLAIN SELECT mem_id, mem_name, addr FROM member WHERE addr = '서울';
```

활동 B

“필요 없는 인덱스” 후보 찾기

각자 SHOW INDEX 결과를 보고, 중복되거나 효과가 낮아 보이는 인덱스를 하나 골라 이유를 설명합니다.

활동 C

정리 질문

“기본키를 문자열로 길게 잡으면 보조 인덱스에는 어떤 영향이 있을까?”

핵심 정리

인덱스는 검색 속도와 관리 비용의 균형이다

- 1 PRIMARY KEY는 InnoDB에서 보통 클러스터형 인덱스가 된다.
- 2 보조 인덱스는 인덱스 키와 기본키 값을 저장한다.
- 3 UNIQUE KEY는 중복 방지와 검색 보조 역할을 함께 한다.
- 4 인덱스가 많으면 쓰기 비용과 저장 공간이 증가한다.
- 5 정렬된 결과가 필요하면 ORDER BY를 사용한다.

참고 문헌: MySQL 8.4 Reference Manual - InnoDB Clustered and Secondary Indexes, CREATE INDEX, DROP INDEX, SHOW INDEX, EXPLAIN